

```

def is_safe(board, row, col, n):
    # Check upper column
    for i in range(row):
        if board[i][col] == 1:
            return False

    # Check upper left diagonal
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check upper right diagonal
    i, j = row - 1, col + 1
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

    return True


def solve_n_queens(board, row, n):
    if row == n:
        return True

    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1

            if solve_n_queens(board, row + 1, n):
                return True

            board[row][col] = 0 # Backtrack

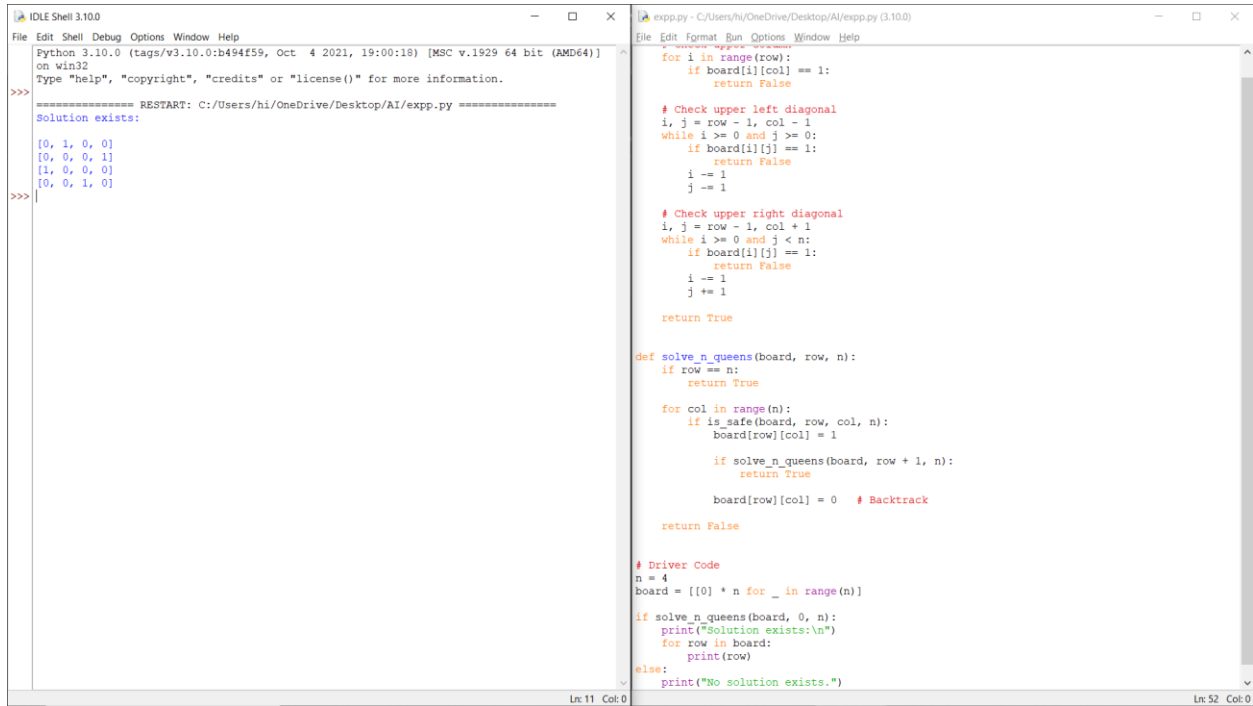
    return False


# Driver Code
n = 4
board = [[0] * n for _ in range(n)]

if solve_n_queens(board, 0, n):
    print("Solution exists:\n")

```

```
for row in board:
    print(row)
else:
    print("No solution exists.")
```



The image shows two side-by-side windows from the Python IDLE 3.10.0 application. The left window is the Shell, and the right window is the editor for a file named 'expp.py'.

Left Window (Shell):

```
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/hi/OneDrive/Desktop/AI/expp.py =====
Solution exists:
[0, 1, 0, 0]
[0, 0, 0, 1]
[1, 0, 0, 0]
[0, 0, 1, 0]
>>>
```

Right Window (expp.py):

```
def is_safe(board, row, col, n):
    # Check upper left diagonal
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check upper right diagonal
    i, j = row - 1, col + 1
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

    return True

def solve_n_queens(board, row, n):
    if row == n:
        return True

    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1

            if solve_n_queens(board, row + 1, n):
                return True

            board[row][col] = 0 # Backtrack

    return False

# Driver Code
n = 4
board = [[0] * n for _ in range(n)]

if solve_n_queens(board, 0, n):
    print("Solution exists:\n")
    for row in board:
        print(row)
else:
    print("No solution exists.")
```